

1 源代码

冒泡排序

```
#include <reg51.h>

void main(){
    unsigned char A[15]={27,5,32,47,38,235,79,17,187,58,23,35,211,104,9};
    unsigned char data *pRam;
    int i=0;
    int j=0;
    unsigned char t;
    pRam=0x30;
    for (i=0;i<15;i++){
        *pRam=A[i];
        pRam++;
    }
    for(i=0;i<14;i++){
        for(j=0;j<14-i;j++){
            if(A[j]>A[j+1]){
                t=A[j];
                A[j]=A[j+1];
                A[j+1]=t;
            }
        }
    }
    for (i=0;i<15;i++){
        *pRam=A[i];
        pRam++;
    }
    while(1);
}
```

选择排序

```
#include <reg51.h>

void main(){
    unsigned char A[15]={27,5,32,47,38,235,79,17,187,58,23,35,211,104,9};
    unsigned char data *pRam;
    int i=0;
    int j=0;
    int index=0;
    unsigned char t;
    pRam=0x30;
    for (i=0;i<15;i++){
        *pRam=A[i];
        pRam++;
    }
    for(i=0;i<14;i++) {
        index=i;
        for(j=i+1;j<15;j++) {
            if(A[j]<A[index]) {
```

```

        index=j;
    }
}
if(index!=i) {
    t = A[i];
    A[i] = A[index];
    A[index] = t;
}
}
for (i=0;i<15;i++){
    *pRam=A[i];
    pRam++;
}
while(1);
}

```

插入排序

```

#include <reg51.h>

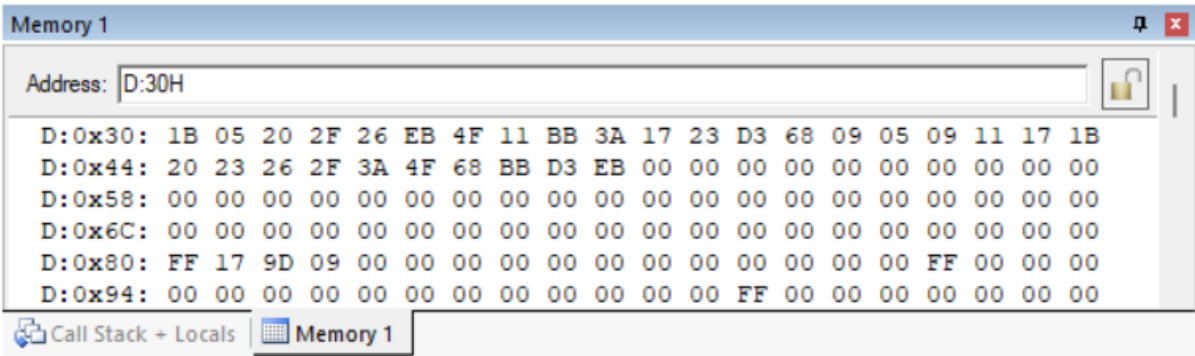
void main(){
    unsigned char A[15]={27,5,32,47,38,235,79,17,187,58,23,35,211,104,9};
    unsigned char data *pRam;
    int i=0;
    int j=0;
    int index=0;
    unsigned char t;
    pRam=0x30;
    for (i=0;i<15;i++){
        *pRam=A[i];
        pRam++;
    }
    for(i=1;i<15;i++){
        {
            if(A[i]<A[i-1])
            {
                t=A[i];
                for(j=i-1;j>=0&& t<A[j];j--)
                {
                    A[j+1]=A[j];
                }
                A[j+1]=t;
            }
        }
    }
    for (i=0;i<15;i++){
        *pRam=A[i];
        pRam++;
    }
    while(1);
}

```

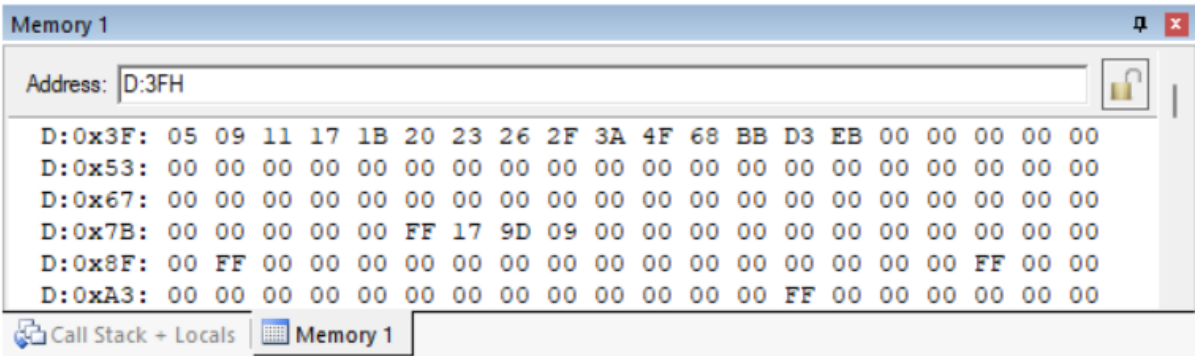
由于内存占用的问题，keil处理递归函数有问题，因此归并排序与快速排序较难实现。

2 调试结果

原数组存到内存RAM 0x30——0x3E



排序后的数组存到内存RAM 0x3F-0x4D



冒泡排序

选择排序

插入排序

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x17
sp_max	0x17
dptr	0x0000
PC \$	C:0x08F6
states	389
sec	0.00038900
+	
psw	0x00

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x17
sp_max	0x17
dptr	0x0000
PC \$	C:0x08F6
states	389
sec	0.00038900
+	
psw	0x00

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x19
sp_max	0x19
dptr	0x0000
PC \$	C:0x08F6
states	389
sec	0.00038900
+	
psw	0x00

3 思考题

1, 本例中采用什么排序算法最合适（时间/空间复杂度）

选用的三种算法都有 $O(n^2)$ 的时间复杂度，三种算法调试时的sp为堆栈栈顶的指针，可以反映定义的局部变量的多少和递归的深度，因此可以反映空间复杂度，冒泡排序和选择排序的空间复杂度均较小；而sec可以反映算法的速度，但是由于数组太小，三者sec相同。归并排序、快速排序和堆排序的时间复杂度都为 $O(n\log n)$ ，均更小，但是所需的局部变量与递归深度更多，有更大的空间复杂度。

对于本例中的数组，选用冒泡排序最好。

2, 考虑8051有限的资源（RAM和算力），如果增加待排序的数组长度，在什么时候是会出现瓶颈（RAM占用和消耗时间快速增加）

内部RAM只有128个字节，经实验，当数组的长度增长到118时，再增加会内存溢出报错，说明还有10个字节用来存放局部变量的。

```
*** ERROR L119: REFERENCE MADE TO ERRONEOUS SEGMENT
SEGMENT: ?C_CS1STARTUP
MODULE: C:\USERS\LENOVO\APPDATA\LOCAL\KEIL_V5\CS1\LIB\CS1S.LIB (?C_CS1STARTUP)
ADDRESS: 0A43H
Program Size: data=129.0 xdata=0 code=596
Target not created.
Build Time Elapsed: 00:00:00
```

数组长度过大的话必须引用外部RAM或者ROM，这会增加访问时间降低效率。同时增大RAM占用。当数组长度接近RAM容量时会出现瓶颈。

3, 有什么方法可以完成大数组排序任务

用归并排序，将数组分块存储到不同寄存器中，分别每块排序，最后将所有数组块合并，与归并排序的思想相同，而且归并排序有更小的时间复杂度 $O(n\log n)$ 。